Instructions: Comments in the Program are just for your understanding do not learn the comments

Slip1:

1. **Write a Java program to print the sum of elements of the array. Also display array elements in ascending order.**

```
//Slip1_1.java
import java.util.*;

public class Slip1_1 {
    public static void main(String[] args) {
        int arr[] = {45, 12, 89, 7, 23, 64};
        int sum = 0;

        // 1. Calculate sum using a simple loop
        for (int i = 0; i < arr.length; i++) {
            sum += arr[i];
        }

        System.out.println("Original Array: " + Arrays.toString(arr));
        System.out.println("Sum of array elements: " + sum);

        // 2. Sort the array in ascending order
        Arrays.sort(arr);

        // 3. Display sorted array elements
        System.out.print("Array elements in ascending order: ");
        for (int i = 0; i < arr.length; i++) {
            System.out.print(arr[i] + " ");
```

```
        }
        System.out.println();
    }
}
```

2. **Design a College/School Registration Form using HTML5 and CSS3 and apply appropriate HTML5 validation attributes.**



//Slip1_2.html

```
<!DOCTYPE html>
<html>
<head><title>Registration Form</title></head>
<body bgcolor="#ffffdd">

<div align="center">
   <div style="width: 600px;" align="left">
      <center>
         <h1>My-Future College / School</h1>
         <p>Please Fill Below Detail In Student Registration Form</p>
      </center>

      <form>
         <b>Student name :</b>
```

```
<input type="text" minlength="3" maxlength="10" required>
<font color="red">Min 3 & Max 10 letter is allow</font><br><br>

<b>Date of birth :</b>
<input type="date" required>
<font color="red">Valid Date is allow</font><br><br>

<b>Mobile no. :</b>
<input type="tel" pattern="[0-9]{10}" required>
<font color="red">Only integer is allow</font><br><br>

<b>Email id :</b>
<input type="email" required>
<font color="red">Valid Email Id is allow</font><br><br>

<b>Password :</b>
<input type="password" minlength="8" required>
<font color="red">Min 8 digit is allow</font><br><br>

<b>Gender :</b>
<input type="radio" name="g" required> Male
<input type="radio" name="g"> Female<br><br>

<b>Hobbies :</b>
<input type="checkbox"> Drawing
<input type="checkbox"> Singing
<input type="checkbox"> Dancing
<input type="checkbox"> Sketch
<input type="checkbox"> Sports<br><br>
```

```
        <b>Course :</b>
        <select required>
            <option value="">---------- Select Admission Course's ----------</option>
            <option value="bcs">B.Sc (CS)</option>
            <option value="bca">BCA</option>
        </select><br><br>

        <b>Student photo :</b> <input type="file" required><br><br>

        <b>Register Time:</b> <input type="time" required><br><br>

        <b>Address :</b><br>
        <textarea rows="3" cols="30" required></textarea><br><br>

        <input type="checkbox" required>
        <b>I hereby declare that above information provided is correct</b><br><br>

        <center>
            <input type="submit" value="Register">
            <input type="reset" value="Reset">
        </center>
    </form>
  </div>
</div>

</body>
</html>
```

Slip2:

1. Write a Java Program to Display Armstrong Numbers Between range. Accept range from user.

An **Armstrong number** is a number that is equal to the sum of its own digits raised to the power of the total number of digits.

3-digit number: Multiply each digit 3 times and add them up.

(Example: $153 = 1^3 + 5^3 + 3^3 = 1 + 125 + 27 = 153$)

4-digit number: Multiply each digit 4 times and add them up.

(Example: $1634 = 1^4 + 6^4 + 3^4 + 4^4 = 1 + 1296 + 81 + 256 = 1634$)

//Slip2_1.java

```
import java.util.Scanner;


public class  Slip2_1
{
   public static void main(String[] args)
 {
       Scanner sc = new Scanner(System.in);
       System.out.print("Enter start limit: ");
       int start = sc.nextInt();


       System.out.print("Enter end limit: ");
       int end = sc.nextInt();

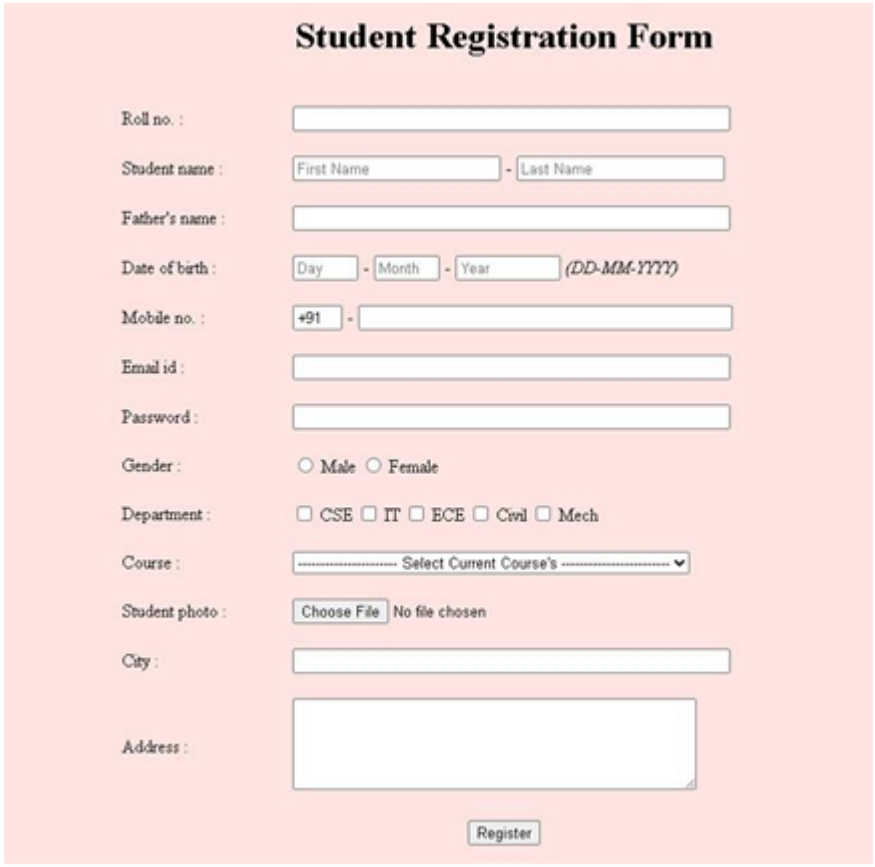

       System.out.println("Armstrong numbers:");
       for (int i = start; i <= end; i++)
{
          int num = i;
          int count = 0;
```

```
            // Step 1: Count total digits
            while (num > 0)
            {
                count++;
                num /= 10;
            }

            // Step 2: Calculate sum of each digit raised to power 'count'
            num = i;
            int sum = 0;
            while (num > 0) {
                int rem = num % 10;
                sum += Math.pow(rem, count);
                num /= 10;
            }

            // Step 3: Check and print
            if (sum == i) {
                System.out.print(i + " ");
            }
        }
    }
}
```

2. Design a Student Registration Form using HTML5 and CSS3 and apply appropriate HTML5 validation attributes.



//Slip2_2.html

```
<!DOCTYPE html>
<html>
<head>
    <title>Student Registration Form</title>
</head>
<body bgcolor="#ffe4e1">

<div align="center">
    <div style="width: 500px;" align="left">
        <center>
            <h1>Student Registration Form</h1>
        </center>

        <form>
```

```
<b>Roll no. :</b><br>
<input type="number" required><br><br>

<b>Student name :</b><br>
<input type="text" placeholder="First Name" required> -
<input type="text" placeholder="Last Name" required><br><br>

<b>Father's name :</b><br>
<input type="text" required><br><br>

<b>Date of birth :</b><br>
<input type="text" placeholder="Day" pattern="0[1-9]|[12][0-9]|3[01]" title="Day (01-31)" size="5" required> -
<input type="text" placeholder="Month" pattern="0[1-9]|1[0-2]" title="Month (01-12)" size="5" required> -
<input type="text" placeholder="Year" pattern="19[0-9]{2}|20[0-2][0-9]" title="Year (4 digits)" size="8" required> <i>(DD-MM-YYYY)</i><br><br>

<b>Mobile no. :</b><br>
<input type="text" value="+91" size="3" readonly> -
<input type="tel" pattern="[0-9]{10}" required><br><br>

<b>Email id :</b><br>
<input type="email" required><br><br>

<b>Password :</b><br>
<input type="password" minlength="8" required><br><br>

<b>Gender :</b>
<input type="radio" name="g" required> Male
```

```
<input type="radio" name="g"> Female<br><br>

<b>Department :</b>
<input type="checkbox"> CSE
<input type="checkbox"> IT
<input type="checkbox"> ECE
<input type="checkbox"> Civil
<input type="checkbox"> Mech<br><br>

<b>Course :</b><br>
<select required>
   <option value="">---------- Select Current Course's ----------</option>
   <option value="btech">B.Tech</option>
   <option value="mtech">M.Tech</option>
   <option value="bca">BCA</option>
</select><br><br>

<b>Student photo :</b><br>
<input type="file" accept="image/*" required><br><br>

<b>City :</b><br>
<input type="text" required><br><br>

<b>Address :</b><br>
<textarea rows="4" cols="30" required></textarea><br><br>

<center>
   <input type="submit" value="Register">
</center>
```

```
        </form>
      </div>
    </div>
    </body>
    </html>
```

Slip5:

1. Write a program to accept a text file from user and display the contents of a file in reverse order and change its case.

```
    //Slip5_1.java
    import java.io.*;
    import java.util.*;

    class Slip5_1
     {
       public static void main(String args[]) throws Exception
     {

          Scanner sc = new Scanner(System.in);

          System.out.print("Enter file name: ");
          String name = sc.nextLine();

          FileReader fr = new FileReader(name);
          String str = "";

          int ch;
          while ((ch = fr.read()) != -1)
             str += (char) ch;
```

```
        fr.close();

        System.out.println("Reverse contents:");

        for (int i = str.length() - 1; i >= 0; i--) {
            char c = str.charAt(i);

            if (Character.isUpperCase(c))
                c = Character.toLowerCase(c);
            else if (Character.isLowerCase(c))
                c = Character.toUpperCase(c);

            System.out.print(c);
        }
    }
}
```

2. Create an image gallery using HTML5 and CSS3 with hover effects, responsive images and Flexbox/Grid layout.

```
<!DOCTYPE html>
<html>
<head>
<title>Image Gallery</title>

<style>
.gallery {
    display: grid;
    grid-template-columns: repeat(4,1fr);
```

```
    gap: 10px;
}
.gallery img {
    width: 100%;
    height: 180px;
    object-fit: cover;
    transition: 0.3s;
}
.gallery img:hover {
    transform: scale(1.1);
}
@media(max-width:600px) {
    .gallery { grid-template-columns: repeat(2,1fr); }
}
</style>
</head>

<body>

<h2>My Image Gallery</h2>

<div class="gallery">
    <img src="nature.jpeg" alt="Nature">
    <img src="mo.jpeg" alt="Mountain">
    <img src="beach.jpeg" alt="Beach">
    <img src="jangle.jpeg" alt="Forest">
</div>

</body>
```

```
</html>
```

Slip 6:

1. Write a program to define a class Account having members custname, accno. Define default and parameterized constructor. Create a subclass called SavingAccount with members savingbal, minbal. Create a derived class AccountDetail that extends the class SavingAccount with members, depositamt and withdrawalamt. Write a appropriate method to display customer details.

//Slip6_1.java

```
 class Account
{
  String custname;
  int accno;
  Account()
 {
  }
  Account(String n, int a)
{
      custname = n;
      accno = a;
  }
}

class SavingAccount extends Account
{
  double savingbal, minbal;
  SavingAccount(String n, int a, double b, double m)
{
      super(n, a);
      savingbal = b;
      minbal = m;
  }
}

class AccountDetail extends SavingAccount
 {
  double depositamt, withdrawalamt;
  AccountDetail(String n, int a, double b, double m, double d, double w)
{
      super(n, a, b, m);
      depositamt = d;
```

```
        withdrawalamt = w;
    }

    void display()
 {
        System.out.println("Customer Name: " + custname);
        System.out.println("Account No: " + accno);
        System.out.println("Saving Balance: " + savingbal);
        System.out.println("Minimum Balance: " + minbal);
        System.out.println("Deposit Amount: " + depositamt);
        System.out.println("Withdrawal Amount: " + withdrawalamt);
    }
}

class Slip6_1
{
    public static void main(String args[])
{
        AccountDetail a = new AccountDetail("Sujata", 101, 10000, 1000, 5000, 2000);
        a.display();
    }
}
```

2. Create a webpage containing a button and link. Change their appearance when the mouse pointer moves over them. (Use :hover Pseudo-Class)

//Slip6_2.html

```
<!DOCTYPE html>
<html>
<head>
<title>Hover Effect</title>

<style>
button {
    padding: 10px;
    margin: 10px;
    background: lightblue;
    text-decoration: none;
}

button:hover, a:hover {
    background: blue;
    color: white;
}
```

```
</style>
</head>

<body>

<button>Click Me</button>
<a href="www.google.com">Visit Link</a>

</body>
</html>
```

Slip7:

1. Write a class Driver with attributes license_no, name, address and age. Initialize values through the parameterized constructor. If age of Driver is less than 18 then user-defined exception should be generated —Age is below 18 years—

//Slip7_1.java

```
import java.util.*;
class AgeException extends Exception
{
   AgeException(String msg)
{
      super(msg);
   }
}

class Driver {
   String license_no, name, address;
   int age;
   Driver(String l, String n, String a, int age)
{
      license_no = l;
      name = n;
      address = a;
      this.age = age;
   }

   void display()
{
try
{
      if (age < 18)
         throw new AgeException("Age is below 18 years");
else
```

```
{
        System.out.println("License No: " + license_no);
        System.out.println("Name: " + name);
        System.out.println("Address: " + address);
        System.out.println("Age: " + age);
    }
}
catch (AgeException e) {
            System.out.println(e.getMessage());
        }
}
class Slip7_1
{
   public static void main(String args[])
 {
        Scanner sc = new Scanner(System.in);
            System.out.print("Enter age: ");
            int age = sc.nextInt();
            Driver d = new Driver("L123", "Rahul", "Nashik", age);
            d.display();
        }

}
```

2. Create an unordered list of Programming Language names and apply different styles to the first and last list items.(Use :first-child and :last-child)

//Slip7_2.html

```
<!DOCTYPE html>
<html>
<head>
<title>Programming Languages</title>

<style>
li:first-child {
   color: red;
}

li:last-child {
   color: blue;
}
</style>
</head>
```

```
<body>

<h2>Programming Languages</h2>

<ul>
  <li>Java</li>
  <li>Python</li>
  <li>C++</li>
  <li>JavaScript</li>
</ul>

</body>
</html>
```

Slip8

1. Write a program to create an abstract class named Shape that contains two integers and an empty method named printArea(). Provide three classes named Rectangle, Triangle and Circle such that each one of the classes extends the class Shape. Each one of the classes contain only the method printArea() that prints the area of the given shape. (use method overriding).

```
//Slip8_1.java
abstract class Shape
{
  int a, b;
  abstract void printArea();
}

class Rectangle extends Shape
{
  void printArea()
 {
    a = 10;
    b = 5;
    System.out.println("Rectangle Area = " + (a * b));
  }
}

class Triangle extends Shape
{
  void printArea()
{
```

```
        a = 10;
        b = 5;
        System.out.println("Triangle Area = " + (a * b / 2));
    }
}

class Circle extends Shape
 {
    void printArea()
{
        a = 5;
        System.out.println("Circle Area = " + (3.14 * a * a));
    }
}

class Slip8_1
{
    public static void main(String args[])
 {
        Shape s;

        s = new Rectangle();
        s.printArea();

        s = new Triangle();
        s.printArea();

        s = new Circle();
        s.printArea();
    }
}
```

2. Create a webpage using JavaScript DOM manipulation and event handling with an image and a button. When the button is clicked, replace the displayed image with another image.

//Slip8_2.html

```
<!DOCTYPE html>
<html>
<head>
<title>Image Change</title>
</head>
<body>
<h2>Change Image</h2>
<img id="img" src="image1.jpg" width="200">
```

```
<br><br>
<button onclick="changeImage()">Change Image</button>
<script>
function changeImage()
{
   document.getElementById("img").src = "image2.jpg";
}
</script>
</body>
</html>
```

Slip9:

1. Write a program which define class Employee with data member as id, name and salary Store the information of 'n' employees and display the name of employee having maximum salary (Use array of object)

```
//Employee.java
import java.util.*;
class Employee
{
   int id;
   String name;
   double salary;

   Employee(int i, String n, double s)
{
      id = i;
      name = n;
      salary = s;
   }

   public static void main(String args[])
{
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter number of employees: ");
      int n = sc.nextInt();
      Employee e[] = new Employee[n];
      for (int i = 0; i < n; i++) {
         System.out.print("Enter id, name and salary: ");
         int id = sc.nextInt();
         String name = sc.next();
         double salary = sc.nextDouble();
         e[i] = new Employee(id, name, salary);
      }
```

```
        int max = 0;
        for (int i = 1; i < n; i++)
       {
           if (e[i].salary > e[max].salary)
               max = i;
        }

        System.out.println("Employee with maximum salary: " + e[max].name);
    }
}
```

2. Write a JavaScript program to demonstrate Arrow Functions for addition, subtraction, multiplication and division.
(Note: const is used because we do not change the function after creating it where let can change the function)

//Slip9_2.html

```
<!DOCTYPE html>
<html>
<body>
<script>
const add = (a,b) => a+b;
const sub = (a,b) => a-b;
const mul = (a,b) => a*b;
const div = (a,b) => a/b;
document.write("Addition = " + add(10,5) + "<br>");
document.write("Subtraction = " + sub(10,5) + "<br>");
document.write("Multiplication = " + mul(10,5) + "<br>");
document.write("Division = " + div(10,5));
</script>
</body>
</html>
```

Slip10:

1. Write a program to read the contents of “abc.txt” file. Display the contents of file in uppercase as output.

//Slip10_1.java

```
import java.io.*;
class Slip10_1
 {
   public static void main(String args[]) throws Exception
 {
```

```
        FileReader f = new FileReader("abc.txt");
        int ch;
        while ((ch = f.read()) != -1)
            System.out.print(Character.toUpperCase((char)ch));
        f.close();
    }
}
```

2. Write a JavaScript program to store student information and marks in variables and generate a formatted student report using template literals(Note: Template literals use backticks (`` below ESC key) and ${variable} to insert values.

//Slip10_2.html

```
<!DOCTYPE html>
<html>
<body>
<script>
const name = "Rahul";
const roll = 101;
const marks = 85;
let report = `
Student Report<br>
----------<br>
Name  : ${name}<br>
Roll No: ${roll}<br>
Marks : ${marks}
`;
document.write(report);
</script>
</body>
</html>
```

Slip11

1. Write a program to create a super class Vehicle having members Company and price. Derive two different classes LightMotorVehicle (mileage) and HeavyMotorVehicle (capacity_in_tons). Accept the information for "n" vehicles and display the information in appropriate form. While taking data, ask user about the type of vehicle first.

//Slip11_1.java

```
import java.util.*;
class Vehicle
 {
   String company;
```

```
    double price;

    Vehicle(String c, double p)
{
        company = c;
        price = p;
    }
}

class LightMotorVehicle extends Vehicle
{
    double mileage;
    LightMotorVehicle(String c, double p, double m)
 {
        super(c, p);
        mileage = m;
    }

    void display()
 {
        System.out.println("Company: " + company);
        System.out.println("Price: " + price);
        System.out.println("Mileage: " + mileage);
    }
}

class HeavyMotorVehicle extends Vehicle
{
    double capacity;
    HeavyMotorVehicle(String c, double p, double cap)
{
        super(c, p);
        capacity = cap;
    }

    void display()
{
        System.out.println("Company: " + company);
        System.out.println("Price: " + price);
        System.out.println("Capacity: " + capacity + " tons");
    }
}
```

```
class Slip11_1
 {
   public static void main(String args[])
{
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter number of vehicles: ");
      int n = sc.nextInt();
      for (int i = 0; i < n; i++) {
         System.out.print("Enter type (1-Light, 2-Heavy): ");
         int type = sc.nextInt();
         System.out.print("Enter company and price: ");
         String c = sc.next();
         double p = sc.nextDouble();
 if (type == 1) {
            System.out.print("Enter mileage: ");
            double m = sc.nextDouble();
            new LightMotorVehicle(c, p, m).display();
         }
else
{
            System.out.print("Enter capacity in tons: ");
            double cap = sc.nextDouble();
            new HeavyMotorVehicle(c, p, cap).display();
         }
      }
   }
}
```

2. Write a JavaScript program using an arrow function to calculate the total and average marks of three subjects. Display the result using template literals

//Slip11_2.html

```
<!DOCTYPE html>
<html>
<body>
<script>
const total = (a, b, c) => a + b + c;
const average = (a, b, c) => (a + b + c) / 3;
const m1 = 80, m2 = 70, m3 = 90;
document.write(`
Total Marks = ${total(m1, m2, m3)}<br>
Average Marks = ${average(m1, m2, m3)}
`);
</script>
```

```
</body>
</html>
```

Slip12:

1. Write a package game which will have 2 classes Indoor & Outdoor. Use a function display () to generate the list of players for the specific game. Use default & parameterized constructor.

Seat_NO/

```
|
├── game/
|   ├── Indoor.java
|   └── Outdoor.java
|
└── Slip12_1.java
```

//Indoor.java

```
package game;
public class Indoor {
   String game;
   Indoor()
 {
      game = "Chess";
   }
   Indoor(String g)
 {
      game = g;
   }
   public void display()
{
      System.out.println("Indoor Game: " + game);
      System.out.println("Players: 2");
   }
}
```

//Outdoor.java

```
package game;
public class Outdoor
 {
   String game;
   Outdoor()
{
      game = "Cricket";
   }
```

```
   Outdoor(String g)
 {
      game = g;
   }

   public void display() {
      System.out.println("Outdoor Game: " + game);
      System.out.println("Players: 11");
   }
}
```

//Slip12_1.java

```
import game.*;

class Slip12_1
{
   public static void main(String args[])
{

      Indoor i = new Indoor("Carrom");
      Outdoor o = new Outdoor("Football");
      i.display();
      o.display();
   }
}
```

2. Write a JavaScript program using an arrow function to calculate the total price of a product. Display the bill using template literals and string interpolation.

//Slip12_2.html

```
<!DOCTYPE html>
<html>
<body>
<script>
const total = (price, qty) => price * qty;
const product = "Pen";
const price = 20;
const qty = 5;
document.write(`
Product : ${product}<br>
Price   : ${price}<br>
Quantity: ${qty}<br>
Total   : ${total(price, qty)}
```

```
`);
</script>
</body>
</html>
```

Slip13:

1. Define a "Clock" class that does the following;

a. Accept Hours, Minutes and Seconds

b. Check the validity of numbers

c. Set the time to AM/PM mode

Use the necessary constructors and methods to do the above task

```
//Clock.java
import java.util.*;
class Clock
 {
   int h, m, s;

   Clock(int h, int m, int s)
 {
      this.h = h;
      this.m = m;
      this.s = s;
   }

   boolean valid()
{
      return h >= 0 && h <= 23 &&
           m >= 0 && m <= 59 &&
           s >= 0 && s <= 59;
   }

   void display()
{
      String ampm;
      int hour;

      if (h < 12)
         ampm = "AM";
      else
         ampm = "PM";

      hour = h % 12;
      if (hour == 0)
```

```
            hour = 12;

        System.out.println("Time: " + hour + ":" + m + ":" + s + " " + ampm);
    }

    public static void main(String args[]) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter hours: ");
        int h = sc.nextInt();

        System.out.print("Enter minutes: ");
        int m = sc.nextInt();

        System.out.print("Enter seconds: ");
        int s = sc.nextInt();

        Clock c = new Clock(h, m, s);

        if (c.valid())
            c.display();
        else
            System.out.println("Invalid Time");
    }
}
```

2. Write a JavaScript program to swap two numbers without using a third variable using array destructuring.

//Slip13_2.html

```
<!DOCTYPE html>
<html>
<body>
<script>
let a = 10;
let b = 20;

[a, b] = [b, a];

document.write("After Swapping:<br>");
document.write("a = " + a + "<br>");
document.write("b = " + b);
</script>
</body>
```

```
</html>
```

Slip 19:

1. Write a Java program to accept a file name from the user and count the number of characters, words, and lines present in the file.

```
//Slip19_1.java
import java.io.*;
import java.util.*;

class Slip19_1
 {
   public static void main(String args[]) throws Exception
{
      Scanner sc = new Scanner(System.in);

      System.out.print("Enter file name: ");
      String file = sc.nextLine();

      BufferedReader br = new BufferedReader(new FileReader(file));

      int characters = 0, words = 0, lines = 0;
      String s;

      while ((s = br.readLine()) != null) {
         lines++;
         characters += s.length();

         String w[] = s.split(" ");
         words += w.length;
      }

      br.close();

      System.out.println("Characters = " + characters);
      System.out.println("Words = " + words);
      System.out.println("Lines = " + lines);
   }
}
```

2. Write a JavaScript program using async/await to simulate user login. If the username and password are correct, display a success message; otherwise, handle the error using try-catch

```
//Slip19_2.html
<!DOCTYPE html>
<html>
<body>
```

```
<script>
async function login(username, password) {
   if (username == "admin" && password == "1234")
      return "Login Successful";
   else
      throw "Invalid Username or Password";
}

async function checkLogin() {
   try {
      let result = await login("admin", "1234");
      document.write(result);
   }
   catch (error) {
      document.write(error);
   }
}

checkLogin();
</script>
</body>
</html>
```

Slip21:

1. Write a program to create parent class College (cno, cname, caddr) and derived class Department (dno, dname) from College. Write a necessary methods to display College details.

//Slip21_1.java

```
class College
 {
   int cno;
   String cname, caddr;

   College(int no, String name, String addr) {
      cno = no;
      cname = name;
      caddr = addr;
   }
}

class Department extends College
{
   int dno;
   String dname;

   Department(int cno, String cname, String caddr, int dno, String dname) {
      super(cno, cname, caddr);
      this.dno = dno;
      this.dname = dname;
   }
```

```
   void display()
 {
      System.out.println("College No: " + cno);
       System.out.println("College Name: " + cname);
       System.out.println("College Address: " + caddr);
       System.out.println("Department No: " + dno);
       System.out.println("Department Name: " + dname);
   }
}

class Slip21_1
 {
   public static void main(String args[])
{
      Department d = new Department(101, "Ashoka College", "Nashik", 1, "Computer
Science");
      d.display();
   }
}
```

2. Write a Node.js program using the path module to perform operations such as joining, resolving and extracting file/directory information.

```
//Slip21_2.js
const path = require("path");
let p = "C:/College/Java/program.js";
console.log("Join:", path.join("C:/College", "Java", "program.js"));
console.log("Resolve:", path.resolve("program.js"));
console.log("Directory:", path.dirname(p));
console.log("File Name:", path.basename(p));
console.log("Extension:", path.extname(p));
```

(Notes: path.join() → joins paths

path.resolve() → gives absolute path

path.dirname() → gives directory

path.basename() → gives file name

path.extname() → gives file extension)

Slip22:

1. Write a program using interface, create a class Product (product_id, product_name, product_cost, product_quantity) default and parameterized constructor. Create objects of class product and display the contents of each object and Also display the object count.

//Product.java

```
interface Display
{
   void display();
}

class Product implements Display
{
   int product_id, product_quantity;
   String product_name;
   double product_cost;
   static int count = 0;
   Product()
{
        count++;
   }

   Product(int id, String name, double cost, int qty)
{
      product_id = id;
      product_name = name;
      product_cost = cost;
      product_quantity = qty;
      count++;
   }

   public void display()
{
      System.out.println("ID: " + product_id);
      System.out.println("Name: " + product_name);
      System.out.println("Cost: " + product_cost);
      System.out.println("Quantity: " + product_quantity);
      System.out.println();
   }

   public static void main(String args[])
 {
      Product p1 = new Product();
      Product p2 = new Product(101, "Pen", 20, 5);
      Product p3 = new Product(102, "Book", 100, 3);
      p1.display();
      p2.display();
      p3.display();
      System.out.println("Object Count = " + count);
```

```
    }
}
```

2. Write a Node.js program using the fs module to perform file operations asynchronously.

//Slip22_2.js

```
const fs = require("fs");
// Write file
fs.writeFile("test.txt", "Hello Node.js", (err) => {
    if (err) throw err;
    console.log("File written");
    // Read file
    fs.readFile("test.txt", "utf8", (err, data) => {
        if (err) throw err;
        console.log("File content:", data);
        // Append file
        fs.appendFile("test.txt", "\nWelcome!", (err) => {
            if (err) throw err;
            console.log("File updated");
        });
    });
});
```

Slip23

1. Write a program for multilevel inheritance such that Country is inherited from Continent. State is inherited from Country. Display the place, State, Country and Continent.

```
//Slip23_1.java
class Continent {
    String continent;

    Continent(String c) {
        continent = c;
    }
}

class Country extends Continent {
    String country;

    Country(String c, String co) {
        super(c);
        country = co;
    }
}

class State extends Country {
```

```
    String state, place;

    State(String c, String co, String s, String p) {
        super(c, co);
        state = s;
        place = p;
    }

    void display() {
        System.out.println("Place: " + place);
        System.out.println("State: " + state);
        System.out.println("Country: " + country);
        System.out.println("Continent: " + continent);
    }
}

class Slip23_1
 {
    public static void main(String args[]) {
        State s = new State("Asia", "India", "Maharashtra", "Nashik");
        s.display();
    }
}
```

2. Write a Node.js program to create, write, read and append data to a file using the fs module.

//Slip23_2.js

```
const fs = require("fs");

// Create and write file
fs.writeFile("data.txt", "Hello Node.js", (err) => {
    if (err) throw err;
    console.log("File created and written");

    // Read file
    fs.readFile("data.txt", "utf8", (err, data) => {
        if (err) throw err;
        console.log("File Content:", data);

        // Append data
        fs.appendFile("data.txt", "\nWelcome to Node.js", (err) => {
            if (err) throw err;
            console.log("Data appended");
        });
```

```
    });
});
```

Slip24:

1. Create an abstract class Shape with methods area & volume. Derive a class Cylinder (radius, height). Calculate area and volume

//Slip24_1.java

```
abstract class Shape
 {
   abstract void area();
   abstract void volume();
}

class Cylinder extends Shape
 {`
   double radius, height;

   Cylinder(double r, double h)
{
      radius = r;
      height = h;
   }

   void area()
{
      double a = 2 * 3.14 * radius * (radius + height);
      System.out.println("Area = " + a);
   }

   void volume()
 {
      double v = 3.14 * radius * radius * height;
      System.out.println("Volume = " + v);
   }
}

class Slip24_1
 {
   public static void main(String args[])
{
      Cylinder c = new Cylinder(5, 10);

      c.area();
      c.volume();
   }
}
```

2. Write a Node.js program to demonstrate synchronous and asynchronous file operations and compare their execution.

//Slip24_2.js

```
const fs = require("fs");
console.log("Start");
// Synchronous
```

```
console.log("Synchronous:");
let data1 = fs.readFileSync("data.txt", "utf8");
console.log(data1);

// Asynchronous
console.log("Asynchronous:");
fs.readFile("data.txt", "utf8", (err, data2) => {
   if (err) throw err;
   console.log(data2);
});

 console.log("End");
```

Slip25:

1. Write a Java program that defines an interface Calculator containing methods add() and subtract() and implements it in class SimpleCalc.

```
//SimpleCalc.java
interface Calculator
{
   void add();
   void subtract();
}

class SimpleCalc implements Calculator
{
   int a = 20, b = 10;

   public void add()
{
      System.out.println("Addition = " + (a + b));
   }

   public void subtract()

      System.out.println("Subtraction = " + (a - b));
   }

   public static void main(String args[])
{
      SimpleCalc c = new SimpleCalc();

      c.add();
      c.subtract();
```

```
    }
}
```

2. Write a Node.js program to perform directory management operations such as creating, listing, renaming and deleting directories. Also demonstrate JSON parsing and writing structured JSON data.

```
//Slip25_2.js
const fs = require("fs");

// Create directory
fs.mkdirSync("MyFolder");
console.log("Directory created");

// List directories
console.log("Directories:", fs.readdirSync("."));

// Rename directory
fs.renameSync("MyFolder", "NewFolder");
console.log("Directory renamed");

// Write JSON data
let student = {
   name: "Rahul",
   age: 20,
   course: "BSc CS"
};

fs.writeFileSync("student.json", JSON.stringify(student, null, 2));
console.log("JSON file written");

// Read and parse JSON
let data = fs.readFileSync("student.json", "utf8");
let obj = JSON.parse(data);
console.log("Student:", obj);

// Delete directory
fs.rmdirSync("NewFolder");
console.log("Directory deleted");
```

Slip26:

1. Define a class Student with data members student_name, student_rollno, total_lectures, attended_lectures. Create an object of Student. If attendance percentage is less than 75%, throw a user-defined exception "Student is Not Eligible for Exam", otherwise display student details.

```
//Student.java
import java.util.*;
class AttendanceException extends Exception
{
   AttendanceException(String msg)
 {
      super(msg);
   }
}

class Student
{
   String student_name;
   int student_rollno, total_lectures, attended_lectures;

   Student(String name, int roll, int total, int attended) {
      student_name = name;
      student_rollno = roll;
      total_lectures = total;
      attended_lectures = attended;
   }

   void display()
 {
      try
      {
      double percentage = (attended_lectures * 100.0) / total_lectures;
      if (percentage < 75)
         throw new AttendanceException("Student is Not Eligible for Exam");
       else
      {
      System.out.println("Name: " + student_name);
      System.out.println("Roll No: " + student_rollno);
      System.out.println("Total Lectures: " + total_lectures);
      System.out.println("Attended Lectures: " + attended_lectures);
      System.out.println("Attendance: " + percentage + "%");
      }
     }
      catch (AttendanceException e)
     {
         System.out.println(e.getMessage());
      }
}
```

```
    public static void main(String args[])
{
    Student s = new Student("Rahul", 101, 100, 80);
      s.display();
    }
  }
```

2. Write a Node.js program using the fs module to copy, rename and delete files asynchronously. Display appropriate success and error messages.

```
//Slip26_2.js
const fs = require("fs");

// Copy file
fs.copyFile("data.txt", "copy.txt", (err) => {
  if (err)
    console.log("Copy Error");
  else {
    console.log("File copied");

    // Rename file
    fs.rename("copy.txt", "newdata.txt", (err) => {
      if (err)
        console.log("Rename Error");
      else {
        console.log("File renamed");

        // Delete file
        fs.unlink("newdata.txt", (err) => {
          if (err)
            console.log("Delete Error");
          else
            console.log("File deleted");
        });
      }
    });
  }
});
```

Slip29:

1. Write a program to accept a number from the user, if number is zero then throw user defined exception "Number is 0" otherwise check whether no is prime or not (Use static keyword)

//Prime.java

```
import java.util.*;
class NumberException extends Exception
{
   NumberException(String msg)
{
      super(msg);
   }
}

class Prime
 {
   static void check(int n)
{
      try {
         if (n == 0)
            throw new NumberException("Number is 0");

         boolean prime = true;

         if (n < 2)
            prime = false;

         for (int i = 2; i < n; i++)
         {
            if (n % i == 0)
              {
               prime = false;
               break;
            }
         }

         if (prime)
            System.out.println("Number is Prime");
         else
            System.out.println("Number is Not Prime");
      }
      catch (NumberException e)
      {
         System.out.println(e.getMessage());
      }
```

```
    }

    public static void main(String args[])
  {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number: ");
        int n = sc.nextInt();
        check(n);
    }
}
```

2. Write a Node.js program to demonstrate file statistics using fs.stat(), including file size, file type, creation/access/modification information, and whether the path represents a file or directory.

```
//Slip29_2.js
const fs = require("fs");
fs.stat("data.txt", (err, stats) => {
    if (err) {
        console.log("Error:", err.message);
        return;
    }

    console.log("File Size:", stats.size, "bytes");
    console.log("Created:", stats.birthtime);
    console.log("Accessed:", stats.atime);
    console.log("Modified:", stats.mtime);

    if (stats.isFile())
        console.log("Type: File");
    else if (stats.isDirectory())
        console.log("Type: Directory");
});
```